We use a Random Forest Regressor, a powerful ensemble method that combines multiple decision trees to make predictions: # Train the model

```
rf_model = RandomForestRegressor(random_state=42) rf_model.fit(X_train, y_train)
```

Step 5: Making Predictions

Once the model is trained, it can predict house prices for the test dataset: # Predictions

```
predictions = rf_model.predict(X_test)
```

Step 6: Evaluating the Model

The Mean Absolute Error (MAE) is used to evaluate the model's performance. MAE measures the average magnitude of errors in predictions, providing a clear indication of model accuracy: # Evaluation

```
mae = mean_absolute_error(y_test, predictions)
print("Mean Absolute Error:", mae)
```

Example Output

For the dataset provided, the output might look like this: Mean Absolute Error: 15000.0

This indicates that the model's predictions are, on average, $15,000 off from the actual prices.

Insights

1. **Model Performance:** The low MAE demonstrates that the Random Forest Regressor effectively captures the relationships between house features and prices.
2. **Feature Importance:** Random forests provide insights into feature importance, highlighting which features most significantly impact the target variable.
3. **Scalability:** The model can be scaled to handle larger datasets with additional features like lot size, year built, and amenities.

Feature Importance

We can examine the importance of each feature in the model: importances = rf_model.feature_importances_

```
for i, feature in enumerate(X.columns):
    print(f"Feature: {feature}, Importance: {importances[i]}")
```